

LazInfos

Das Lazarus Beispielbuch

Andreas Frieß
Die Community von www.lazarusforum.de

19. Januar 2025

Copyright (C) 2007-2025 Andreas Frieß. Es wird die Erlaubnis gewährt, dieses Dokument zu kopieren, zu verteilen und/oder zu modifizieren, unter den Bestimmungen der GNU Free Documentation License, Version 1.2 oder jede spätere Version, veröffentlicht von der Free Software Foundation; mit keinen unveränderlichen Abschnitten, keinen vorderen Umschlagtexten und keinen hinteren Umschlagtexten. Eine Kopie der Lizenz ist aufgenommen in den Abschnitt mit dem Titel "GNU Free Documentation License".

Copyright © 2007-2025 Andreas Friess Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 1.2 or any later version published by the Free Software Foundation; with no Invariant Sections, no Front-Cover Texts, and no Back-Cover Texts. A copy of the license is included in the section entitled “GNU Free Documentation License”.

Inhaltsverzeichnis

1	Vorwort	5
1.1	Danksagung	5
2	Grundlagen Datenbank	6
2.1	Datenbanken	6
2.1.1	Einführung in die Datenbanktheorie	6
	Begriffe	6
	Was ist eine Datenbank	6
	Desktop und Client-Server Datenbankarten	7
	Relationale Datenbanken	8
	Grunddaten	10
2.1.2	DDL Datendefinitionssprache	12
2.1.3	DML Datenveränderungssprache	12
	SELECT	12
	Beispiele zu SELECT	14
	INSERT	16
	Beispiele zu INSERT	17
	UPDATE	17
	Beispiele zu UPDATE	18
	DELETE	18
	Beispiele zu DELETE	19
2.1.4	DCL Datenkontrollsprache	19
3	Lazarus IDE	21
3.1	Entwicklungsumgebung	21
3.1.1	Tastenkombinationen	21
3.1.2	Fehlersuche	23
	DebugLn	24
	Heap Trace	24
3.1.3	Projektschablonen	25
	Beschreibung	25
	Beispiel	26
3.2	GNU Free Documentation License	29
	GNU Free Documentation License	29
	1. APPLICABILITY AND DEFINITIONS	29
	2. VERBATIM COPYING	31

Inhaltsverzeichnis

3. COPYING IN QUANTITY	31
4. MODIFICATIONS	31
5. COMBINING DOCUMENTS	33
6. COLLECTIONS OF DOCUMENTS	34
7. AGGREGATION WITH INDEPENDENT WORKS	34
8. TRANSLATION	34
9. TERMINATION	35
10. FUTURE REVISIONS OF THIS LICENSE	35
ADDENDUM: How to use this License for your documents	35

1 Vorwort

1.1 Danksagung

Meiner Frau Vor allen möchte ich meiner Frau danken. Für ihre schier endlose Geduld mit mir, beim Schreiben dieses Buches.

Mitwirkende carli für das Korrekturlesen,...

2 Grundlagen Datenbank

2.1 Datenbanken

2.1.1 Einführung in die Datenbanktheorie

Das Folgende Kapitel wendet sich speziell an den Personenkreis der in die Theorie von Datenbanken noch nicht so eingedrungen ist, beziehungsweise dient als Nachschlagwerk für die verschiedenen Begriffe. Man kann also bei entsprechenden Vorwissen die folgenden erklärenden Kapitel überspringen und bei Bedarf nachschlagen.

Begriffe

Um Missverständnisse auszuschließen und zu einer gemeinsamen Sprachregelung zu kommen, sollte man die verwendeten Begriffe möglichst genau definieren:

- Eine Datenmenge ist eine Menge von einzelnen Datensätzen. Jeder Datensatz besteht aus mindesten einem Feld. Die Herkunft dieser Datenmenge ist nicht festgelegt.
- Eine Tabelle ist als Datenbankbestandteil eine spezielle Ausführung einer Datenmenge. Die Tabelle speichert als physisches vorhandenes Element die Daten.
- Eine Abfrage ist eine virtuelle Datenmenge, die den Inhalt von tatsächlich vorhandenen Tabellen in einer frei wählbaren Anordnung abbildet, manchmal auch Projektion genannt.
- Eine Datenbank ist eine Zusammenstellung von logisch zusammengehörigen Tabellen.
- Ein Datenbankserver verwaltet verschiedene Datenbanken und stellt auch das Management, die Sicherung und andere Verwaltungsdienste zur Verfügung.

Wichtige Informationen bei der Entwicklung einer Datenbankanwendung sind das Verhalten der Datenbank bzw. des Datenbankservers selbst.

Was ist eine Datenbank

Eine Datenbank ist eine geordnete Sammlung von Daten, die auf irgendeine Weise miteinander in Beziehung stehen.

Die ersten Generationen von Datenbanken waren sogenannte File-Systeme. Zuerst auf Band, dann auch auf Festplatten. In diesen Datei-Systemen wurden die Daten nacheinander abgespeichert. Um auf einen bestimmten Datensatz zu zugreifen, muss man an den Anfang der Datei (oder Bandes) gehen und anschließend alle Datensätze

durchlaufen, bis man den richtigen gefunden hat. Somit ist auch klar, dass ein einfaches Sortieren oder Einfügen von Daten in eine sortierte Datenmenge enormen Aufwand und Kapazität erfordert hat. Um diesen Beschränkungen zu entfliehen, (und durch neue, schnellere und größere Festplatten ermöglicht) haben sich aus diesen Systemen die heutigen relationalen oder objektorientierten Datenbanken entwickelt. Die derzeit am meisten verwendeten Datenbanken sind die relationalen und im weiteren werde ich nur noch diese behandeln.

Desktop und Client-Server Datenbankarten

Gerade der Begriff Netzwerkdatenbank ist sehr verwirrend. Wird damit eine Access-Datenbank, die mehrere Benutzer über das Netzwerk verwenden so bezeichnet? Oder eine Serverbasierende Datenbank? Die folgenden Erklärungen sollten die Begriffe klarer werden lassen.

Stand-Alone Datenbank Eine Stand-Alone Datenbank ist eine Desktop-Datenbank, es befinden sich daher die Daten auf dem Arbeitsplatzrechner. Auf die Daten kann immer nur ein Anwender, mit immer nur einem Programm zugreifen. Es ist zwar prinzipiell möglich über ein Netzwerk auf die Datenbankdatei zuzugreifen, aber es kann der eine nur in der Datenbank arbeiten, wenn der andere sein Programm geschlossen hat. Probleme die durch den gleichzeitigen Zugriff entstehen können daher gar nicht auftreten. Bei jeder etwas umfangreicheren Datenbank wird dieses Verhalten zu einem Engpass. Man stelle sich nur vor, der eine Benutzer öffnet die Datenbankdatei und vergisst auf das Schließen des Programms. Kein anderer Benutzer kann die Daten in der Zwischenzeit benutzen!

File-Share Datenbank Moderne Netzwerke bieten die Möglichkeit, dass mehrerer Anwender auf ein und dieselbe Datei zugreifen. Auf diese Weise ist es auch möglich dass mehrere Programme auf ein und dieselbe Datenbankdatei zugreifen. Diese Version der Desktop-Datenbank nennt man File-Share Datenbank und damit ist bereits ein echter Mehrbenutzer-Betrieb möglich. Das ganze hat jedoch (unter anderem) einen entscheidenden Nachteil: Die Datenverarbeitung erfolgt auf den Arbeitsplatzrechnern. Für Abfragen muss der jeweilige ganze Datenbestand zu den Arbeitsplatzrechnern gebracht werden, dementsprechend hoch ist die Belastung für das Netzwerk. Weiter können auch Störungen am Netzwerk leicht zu Datenverlust bzw. zu Inkonsistenz der Datenbeständen führen.

Client-Server Datenbank Bei Client-Server Datenbanken hat nur der Datenbankserver selbst direkten Zugriff auf die Dateien des Datenbestandes. Anfragen werden somit direkt an den Datenbankserver gestellt, von diesem bearbeitet und die Ergebnisse an den Arbeitsplatzrechner zurückgeliefert. Die Verwaltung beim gleichzeitigen Zugriff durch mehrere Arbeitsplatzrechner obliegt dem Datenbankserver. Falls eine Verbindung durch eine Störung abbricht, so wird dieses erkannt und die noch nicht kompletten Anfragen verworfen und somit die Konsistenz der Daten erhalten. Gerade zur File-Share Datenbank können Netzbelastungen drastisch gesenkt werden. Man stelle sich nur vor, dass man den größten Wert aus einer unsortierten Tabelle mit 1 Millionen Datensätzen haben will. Bei

der File-Share Datenbank müssen alle Datensätze geholt und bearbeitet werden, bei der Client Server Datenbank nur das Ergebnis. Weitere Bereiche sind die Möglichkeit Backups zu erstellen während die Datenbank weiter in Verwendung ist.

Relationale Datenbanken

Der Begriff relationale Datenbank geht auf einen Artikel von E. F. Codd zurück, der 1970 veröffentlicht wurde. Codd bezeichnet Datenbanken als "minimal relational", wenn sie folgende Bedingungen erfüllen. [list] [*]Die Informationen werden einheitlich in Form von Tabellen repräsentiert. [*]Der Anwender sieht keine Verweisstrukturen zwischen den Tabellen. [*]Es gibt mindestens die Operation der Selektion, der Projektion und des JOIN definiert. [/list] 1985 veröffentlichte Codd zwölf Regeln, die relationalen Datenbanken im strengerem Sinn definieren, Ende 1990 veröffentlichte er ein Buch über relationale Datenbanken, in dem er die einstigen zwölf Regeln des relationalen Modells auf 333 Regeln differenziert. Dadurch wird die Relationalität einer Datenbank bis ins letzte Detail festgeschrieben. Soweit die Theorie, normalerweise sprechen Hersteller von relationalen Datenbanken, wenn die Mehrheit der 12 Regeln eingehalten werden.

Begriffe in relationalen Datenbanken Man kann nicht über relationale Datenbanken sprechen, ohne zuvor einige Begriffe zu klären.

Relationen Eine Relation ist gleich einer Tabelle. Die Daten werden daher in Relationen gespeichert.

Attribut und Tuples Die Attribute sind die Spalten einer Relation (Tabelle), die Tuples sind die Datensätze.

Degree und Kardinalität Die Zahl der Attribute einer Relation nennt man Degree, das ist der Ausdehnungsgrad. Die Zahl der Tuples ist die Kardinalität. Eine Relation mit Degree Null macht keinen Sinn, eine Kardinalität von NULL Tuples hingegen ist eine leere Relation.

Domain Eine Domain ist ein Wertebereich. Man kann z.B. eine Domain Nachnamen vom Type „Zeichen mit Länge 20“ erstellen. Diese wird immer dann verwendet wenn man Datenspalten mit dem Type „Nachname“ erstellen muss. Warum dieser Umweg? Wenn man später Tabellen miteinander verknüpft (in Relation bringt), so müssen die Spalten (Attribute) der gleichen Domain unterliegen. Habe ich vorher eine Domain definiert, so gibt es keine Probleme. Weiter ist es kein Problem wenn man draufkommt das die „Nachnamen“ länger sind, so kann die Definition der Domain geändert werden und alle Spalten haben die richtige Länge. Würde ich beim händischen Editieren eine Spalte vergessen so würde die Datenbank nicht mehr korrekt arbeiten.

NULL Ein „Nichtwert“ der anfangs immer für Verwirrung sorgt ist NULL. NULL ist nicht gleich Null! Ein besserer Ausdruck wäre UNBEKANNT. NULL macht übrigens aus einer zweiwertigen Logik (Ja/Nein) eine dreiwertige (Ja/Nein/Unbekannt). Vorstellen kann man es sich am besten mit einem Beispiel. Jedem Konferenzraum kann ein Beamer mit seiner Nummer zugeteilt werden. Die Räume welche keinen Beamer besitzen (zuwenige Beamer vorhanden) bekommen als Wert NULL zugeteilt, da ja ein nicht vorhandener Beamer auch keine Nummer besitzt, folglich also unbekannt ist.

Schlüssel Im Zuge der Normalisierung (welche später erklärt wird) werden die Daten auf viele Tabellen verteilt. Um dieses Tabellen wieder richtig in Relation zu bringen werden verschiedene Schlüssel, auch Keys genannt, verwendet.

Primärschlüssel (Primary Key) Jede Relation (Tabelle) besitzt einen Primärschlüssel um einen Datensatz eindeutig zu identifizieren. Ausnahmen von dieser Regel gibt es nur bei „M:N Verknüpfungen“ für die Zwischentabellen verwendet werden (die meisten Datenbanksysteme können diese Verknüpfungen nicht direkt abbilden). Ein Primärschlüssel ist immer eindeutig und ohne Duplikate. Meistens wird dafür eine fortlaufende Nummer verwendet, ist aber nicht zwingend. Vorsicht bei bestimmten Fällen, denn selbst Sozialversicherungsnummern müssen nicht eindeutig sein !

Sekundärschlüssel (Secondary Keys) Werden dafür verwendet um bei bestimmten Datenbankoperationen die Effizienz zu steigern, da die Datensätze intern nicht ungeordnet sondern in sortierter Reihenfolge verwaltet werden. Beim verwenden sollte aber immer auf die zugrunde liegende Datenbank Rücksicht genommen werden, da die Vor- und Nachteile stark datenbankabhängig sind.

Fremdschlüssel (Foreign Key) Ist der Verweis in der Tabelle auf einen Primärschlüssel in einer anderen Tabelle. Gerade in relationalen Datenbanken gibt es sehr viele Verknüpfungen die auf Primär- und Fremdschlüsselpaaren aufbauen. Wichtig ist, das Primär- und Fremdschlüssel der gleichen Domain unterliegen.

Referentielle Integrität Die referentielle Integrität stellt sicher das die Daten zueinander (über Primär- und Fremdschlüssel) glaubhaft bleiben. Ein einfügen, ändern oder löschen ist zu verweigern wenn dadurch die Datenintegrität verletzt würde. Man kann zum Beispiel keine Datensätze aus der Personentabelle löschen, solange in der Tabelle der Bestellungen auf diese Personen verwiesen wird.

Normalisierung Unter der Normalisierung einer Datenbank, wird die Technik des logischen Datenbankdesigns bezeichnet. Es erfolgt meistens durch das Schrittweise optimieren der Datenbank zur Designzeit. Theoretiker haben insgesamt 5 Stufen der Normalisierung herausgearbeitet, wobei in der Praxis meist nur die ersten 3 Stufen verwendet werden.

Warum wird meistens nur bis zur 3. Normalform normalisiert? Bei der Anwendungsentwicklung besteht meistens nicht das Ziel, möglichst den exakten theoretische Grundlagen zu entsprechen,

2 Grundlagen Datenbank

Ausgangslage	Alle Informationen in einer Tabelle
1. Normalform	Jede Spalte einer Tabelle enthält unteilbare Informationen. Die Datensätze verwenden keine sich wiederholenden Informationen, die nicht auch zu einer separaten Gruppe zusammengefasst werden könnten
2 Normalform	Es wird die 1. Normalform eingehalten und alle Informationen in nicht Schlüsselfeldern hängen nur vom kompletten Primärschlüssel ab
3 Normalform	Es wird die 2 Normalform eingehalten und alle Informationen in den nicht Schlüsselfeldern sind untereinander nicht abhängig.
4 Normalform	Es wird die 3. Normalform eingehalten und in gleichen Tabellen sind keine unabhängigen Objekte vorhanden, zwischen denen eine m:n Beziehung bestehen könnte
5 Normalform	Die normalisierte Datenbank kann nicht weiter in Tabellen mit weniger Attributen konvertiert werden. Es muss sich jederzeit der unnormalisierte Ursprungszustand ohne Informationsverlust herstellen lassen

Tabelle 2.1: Datenbank Normalisierungsstufen Übersicht

sondern eine möglichst effiziente Komplettlösung für das Problem zu erhalten. Dazu gehört natürlich auch, die Rücksicht auf das verwendete Datenbanksystem und die damit zu erzielenden Performance. Es leuchtet jedem ein, dass die Aufteilung der Informationen auf viele Tabellen bei einer Auswertung zu schlechteren Ergebnissen führt. Somit kann es sein, dass in Teilbereichen sogar eine Denormalisierung aus Performancegründen nötig ist, oder der gewonnene Platz bzw. das Geschäftsmodell keine Normalisierung sinnvoll erscheinen lassen.

Grunddaten

Mit Grunddaten werden die Informationen bezeichnet, die Voraussetzung für die tägliche Arbeit sind und während des täglichen Betriebs anfallen. Sie stellen die Basis des Datenbanksystems dar. Die Grunddaten werden in zwei Kategorien, Stammdaten und Bewegungsdaten, eingeteilt.

Stammdaten Stammdaten sind diejenigen Grunddaten, die über einen längeren Zeitraum benötigt werden. Sie bilden den Grundbestand an Daten für das Datenbanksystem und werden auch als Bestandsdaten bezeichnet. Stammdaten weisen eine geringe Änderungshäufigkeit auf. Üblicherweise ist bei der Neuanlage von Stammdaten noch nicht bekannt, wann Änderungen zu erwarten sind und wie lange die Daten insgesamt gültig sind. Da auf Stammdaten häufig zugegriffen wird, ist ihre aktuelle Pflege notwendig, so dass Änderungen unmittelbar im Datenbestand nachgezogen werden sollten. Somit ist auch die normale Zugriffsart

festgelegt, auf Stammdaten wir meistens in Verbindung mit Abfragen lesend zugegriffen, nur die Wartung erfolgt schreibend.

Bewegungsdaten Im Gegensatz zu Stammdaten haben Bewegungsdaten eine begrenzte Lebensdauer, die durch einen vorgegebenen Lebenszyklus beschrieben ist. Bewegungsdaten haben einen konkreten Zeitbezug, der für die Bedeutung und Interpretation der Information wichtig ist. Des weiteren beziehen sich Bewegungsdaten auf Stammdaten, weshalb sie auch als abgeleitete Daten bezeichnet werden. Da Bewegungsdaten gegenüber Stammdaten in der Menge mächtiger sind, ist gerade hier auf ein gutes Design zu achten. Betrachten wir es am Beispiel eines Zählers einer Station: Die Zähler werden im 10 Minutenrhythmus in die Bewegungsdaten eingefügt, das sind 144 Datensätze pro Tag, währenddessen der Stammdatenteil gleich bleibt, nämlich 1 Datensatz.

Author: Andreas Frieß Lizenz: GFDL

2.1.2 DDL Datendefinitionssprache

Die Datendefinitionssprache (Data Definition Language = DDL¹) umfasst die Sprachteile von Datenbanksprache, mit deren Hilfe man Datenstrukturen wie Tabellen und andere ähnliche Elemente erzeugt. Es sind das die Befehle die mit *CREATE* beginnen, zum Beispiel *CREATE TABLE* und *CREATE INDEX*.

2.1.3 DML Datenveränderungssprache

Die Datenveränderungssprache (Data Manipulation Language = DML²) umfasst die Sprachteile von Datenbanksprache, die sich mit dem Lesen, Ändern und Löschen von Daten beschäftigen. Es sind das die Befehle *SELECT*, *INSERT*, *UPDATE* und *DELETE*.

Im folgend sehen wir uns die wichtigsten Befehle an, wobei ein Befehl besonders mächtig ist—*SELECT*—.

SELECT

SELECT ist einer der Vielseitigsten und am meisten eingesetzten Befehle überhaupt. Er dient zum Abfragen von Datenmengen aus der Datenbank. Er ermöglicht die Abfrage von Daten aus den Tabellen (die Projektion)

Einfachste Darstellung Bei der einfachsten Abfrage haben wir es mit wenigen Schlüsselwörter zu tun.

```
SELECT [DISTINCT] 'Auswahlliste'
FROM 'Quelle'
WHERE 'Where-Klausel'
[GROUP BY ('Group-by-Attribut')+
[HAVING 'Having-Klausel']]
[ORDER BY ('Sortierungsattribut'' [ASC|DESC])+];
```

Mit *SELECT* wird die Abfrage eingeleitet, anschliessend kommt die Auswahlliste der gewünschten Tabellenspalten. Dann das *FROM*, das angibt welche Tabellen die Daten bereitstellen und das *WHERE*, das angibt nach welchen Kriterien die Daten vorselektiert werden.

SELECT 'Auswahlliste' Hier gibt man die einzelnen Tabellenspalten an, die sich später in der rückgelieferten Datenmenge finden. Sind die Name nicht eindeutig genug, besonders wenn mehrere Tabellen betroffen sind, so muß man den Tabellennamen und eventuell auch die Datenbank angeben, sprich die Eindeutigkeit qualifizieren. Somit kann ein *SELECT* so aussehen: *SELECT_MyDB.STPerson.Vorname*, meistens wird die verkürzte Schreibweise verwendet wie *SELECT_Vorname*.

¹siehe auch http://de.wikipedia.org/wiki/Data_Definition_Language

²siehe auch http://de.wikipedia.org/wiki/Data_Manipulation_Language

In komplexeren Abfragen kann auch statt dem Tabellennamen alleine, eine Funktion stehen. Damit kann man in Datenmengen mathematische und statistische Funktionen verwenden. Auch Verzweigungen und Berechnungen sind möglich. Zu diesen Punkten kommen dann an späterer Stelle die entsprechenden Erklärungen.

DISTINCT erzwingt die Vermeidung von doppelten Datensätzen. Es überspringt in der Ausgabe die mehrfach vorkommenden Datensätze. Die Folge ist ein höherer Aufwand am Server, da das ursprüngliche Ergebnis (ohne DISTINCT) meist noch entsprechend nachbearbeitet werden muß. Bevor man DISTINCT zur Korrektur unerklärlicher doppelter Datensätze verwendet (Verschleiern von Problemen), sollte man sich zuerst seine JOINS und WHERE Klauseln ganz genau ansehen, denn dort liegt meistens das Problem.

Eine gerne verwendete Abkürzung stellt das beliebte **SELECT_*** dar. Hier darf dann das Programm zur Laufzeit erraten, welche Spalten es gibt und von welchen Typ sie sind. Es einfach zu verwenden, birgt aber in fertigen Programmen einiges an versteckten Fehlerquellen. Wird die dem SELECT zugrunde liegende Tabelle geändert oder auch nur Spalten getauscht, so kann es sein, daß Fehler nicht beim ausführen des Statements auffallen (Spalte xx nicht gefunden) und erst andere Programmteile dann unklare Fehlermeldung produzieren. Ich empfehle deshalb, die Spaltennamen wirklich anzugeben. Es gibt Programme die gerade diese Eigenschaft ausnutzen, dort wird es aber bewusst gemacht und entsprechend abgesichert.

FROM 'Quelle' FROM gibt die Quelle der Daten an. Es ist einfach der Tabellename, eine Verbindung von Tabellen (JOIN) oder auch eine untergelagerte SELECT Anweisung.

WHERE 'Where-Klausel' Die WHERE - Klausel schränkt die Ergebnisdatenmenge ein. Bei einfachen Beispielen wird sie gerne, oft zu Unrecht, weggelassen. Wir müssen aber immer Datenmengen betrachten, das Ergebnis kann eine Zeile oder eine million Zeilen sein. Wenn wir in einer Adressdatenbank eine Personen suchen, so wird es trotzdem sinnvoll sein, von Haus aus die Suche einzuschränken. Denn alles was nicht an Daten sinnlos übertragen werden muß, spart Bandbreite, Speicher und erhöht die Geschwindigkeit.

GROUP BY ('Group-by-Attribut') Die Daten werden nach dem Group-by-Attributen zusammengefasst (gruppiert). Dazu müssen auch in der Auswahlliste, für alle nicht durch GROUP BY erfassten Spalten, entsprechende Operationen verwendet werden (SUM, AVG, MIN, ...).

HAVING 'Having-Klausel' Ist als ein WHERE zu betrachten, das allerdings erst **nach** dem Gruppieren wirksam ist und deshalb nur auf die Gruppierungsfunktionen wirksam ist.

ORDER BY ("Sortierungsattribut" [ASC|DESC]) Die ausgegeben Daten werden entsprechend sortiert. Das Sortierungsattribut sind die entsprechenden Spaltennamen in der reihenfolge wie sortiert werden soll.

Beispiele zu SELECT

*SELECT * FROM st_person;*

Ohne Einschränkung oder Sortierung.

```
"STPerson","cVName","cFName","cMName","cRName"
378,"Hope","Giordano","HoGi","Hope0Giordano"
379,"Rose","Bruno","RoBr","Rose4Bruno"
380,"Lauren","Morgan","LaMo","Lauren4Morgan"
381,"Megan","Coleman","MeCo","Megan9Coleman"
```

*SELECT * FROM st_person where STPerson = 875;*

Die Person deren ID 875 ist.

```
"STPerson","cVName","cFName","cMName","cRName"
875,"Hannah","Collins","HaCo","Hannah3Collins"
```

*SELECT * FROM st_person where cVName like 'H%';*

Alle Personen deren Vorname mit „H“ beginnt. Das Prozentzeichen „%“ ist eine Wildcard. So wie bei manchen Betriebssystemen der Stern „*“.

```
"STPerson","cVName","cFName","cMName","cRName"
875,"Hannah","Collins","HaCo","Hannah3Collins"
406,"Hannah","Cook","HaCo","Hannah9Cook"
845,"Hannah","Doyle","HaDo","Hannah9Doyle"
1363,"Hannah","Foster","HaFo","Hannah6Foster"
```

*SELECT * FROM st_person order by cFName;*

Alle Personen, aber nach Familienname aufsteigend sortiert.

```
"STPerson","cVName","cFName","cMName","cRName"
1258,"Caitlin","Adams","CaAd","Caitlin4Adams"
687,"Taylor","Alexander","TaAl","Taylor5Alexander"
644,"Renee","Alexander","ReAl","Renee8Alexander"
885,"Taylor","Alexander","TaAl","Taylor3Alexander"
1131,"Sarah","Alexander","SaAl","Sarah5Alexander"
603,"Megan","Allen","MeAl","Megan0Allen"
1172,"Isabella","Allen","IsAl","Isabella0Allen"
472,"Isabelle","Allen","IsAl","Isabelle6Allen"
```

*SELECT * FROM st_person order by cFName desc;*

Alle Personen, aber nach Familienname absteigend sortiert.

```
"STPerson","cVName","cFName","cMName","cRName"
842,"Isabella","Young","IsYo","Isabella6Young"
1232,"Taylor","Young","TaYo","Taylor3Young"
427,"Ashley","Young","AsYo","Ashley3Young"
420,"Kyla","Young","KyYo","Kyla2Young"
668,"Alexandra","Wright","AlWr","Alexandra6Wright"
1070,"Madison","Wright","MaWr","Madison5Wright"
```

2 Grundlagen Datenbank

*SELECT * FROM st_person where cVName like 'H%' order by cFName;*

Alle Personen deren Vorname mit „H“ beginnt und nach Familienname aufsteigend sortiert.

```
"STPerson","cVName","cFName","cMName","cRName"
932,"Hope","Bell","HoBe","Hope3Bell"
1194,"Harmony","Campbell","HaCa","Harmony7Campbell"
515,"Harmony","Clark","HaCl","Harmony4Clark"
1279,"Holly","Collins","HoCo","Holly7Collins"
875,"Hannah","Collins","HaCo","Hannah3Collins"
406,"Hannah","Cook","HaCo","Hannah9Cook"
1296,"Harmony","Diaz","HaDi","Harmony1Diaz"
```

SELECT cVName, cFName FROM st_person where cVName like 'H%' order by cFName;

Anzeige nur der Spalten „cVName“ und „cFName“ und alle Personen deren Vorname mit „H“ beginnt und nach Familienname aufsteigend sortiert.

```
"cVName","cFName"
"Hope","Bell"
"Harmony","Campbell"
"Harmony","Clark"
"Holly","Collins"
"Hannah","Collins"
"Hannah","Cook"
"Harmony","Diaz"
```

SELECT cVName, cFName FROM st_person where (cVName like 'H%') or (cVName like 'A%') order by cFName;

Anzeige nur der Spalten „cVName“ und „cFName“ und alle Personen deren Vorname mit „H“ oder „A“ beginnt und nach Familienname aufsteigend sortiert.

```
"Alyssa","Butler"
"Abby","Butler"
"Ashley","Butler"
"Ashley","Byrne"
"Harmony","Campbell"
"Harmony","Clark"
"Anna","Clark"
"Abby","Coleman"
"Hannah","Collins"
"Amelia","Collins"
"Anna","Collins"
```

SELECT count(cVName), cVName FROM st_person group by cVName order by count(cVName) desc; Anzahl der Vorkommen der Vornamen absteigend sortiert.

```
"count(cVName)","cVName"
19,"Amelia"
19,"Angel"
17,"Laura"
16,"Elizabeth"
16,"Paris"
16,"Ruby"
15,"Caitlin"
14,"Sophie"
14,"Abby"
14,"Mikayla"
```

SELECT count(cVName) as Anzahl, cVName as Vorname FROM st_person group by cVName order by count(cVName) desc; Anzahl der Vorkommen der Vornamen absteigend sortiert. Dazu noch die Spaltennamen geändert.

```
"Anzahl","Vorname"
19,"Amelia"
19,"Angel"
17,"Laura"
16,"Elizabeth"
16,"Paris"
16,"Ruby"
15,"Caitlin"
14,"Sophie"
14,"Abby"
14,"Mikayla"
```

SELECT count(cVName) as Anzahl, cVName as Vorname FROM st_person group by cVName desc having Anzahl = 16; Anzahl der Vorkommen der Vornamen, Dazu noch die Spaltennamen geändert und die Anzahl muß sechzehn sein

```
"Anzahl","Vorname"
16,"Elizabeth"
16,"Paris"
16,"Ruby"
```

INSERT

INSERT wird für das Einfügen von Datensätzen in eine Tabelle verwendet.

Einfache Darstellung Beim einfachen Einfügen haben wir es mit wenigen Schlüsselwörter zu tun.

```
INSERT 'Ziel' ('Spaltenliste')
VALUES ('Werteliste')
```


Bei dem Ziel handelt es sich um die Tabelle in die die Daten eingefügt werden sollen. Die Spaltenliste kann auch weggelassen werden, wenn die Werteliste in der exakt gleichen Reihenfolge ist, wie die Definition in der Tabelle.

Wenn die Spaltenliste vorhanden ist, so müssen die Werte in der Werteliste in der gleichen Reihenfolge stehen. Es muß dann aber nicht die Reihenfolge und Anzahl der Spalten mit der Tabellen Definition übereinstimmen. Das wird normalerweise verwendet, da an Spalten mit automatischen Zählern (meist Primärschlüssel) keine Werte übergeben werden dürfen!

Beispiele zu INSERT

INSERT st_person (cVName,cFName,cMName, cRName) VALUES ("Hope","Giordano","HoGi","Hope0Giordano"); Bei *INSERT* und anderen Befehlen zur Erstellung von Daten, gibt es kein Ergebnismenge.

```
SELECT * FROM st_person;
```

```
"STPerson","cVName","cFName","cMName","cRName"
1,"Hope","Giordano","HoGi","Hope0Giordano"
2,"Harmony","Campbell","HaCa","Harmony7Campbell"
3,"Harmony","Clark","HaCl","Harmony4Clark"
4,"Holly","Collins","HoCo","Holly7Collins"
```

Die Spalte STPerson, ist zwar beim INSERT nicht angegeben, wird vom System automatisch vergeben.

Ein weiteres Beispiel befindet unter Projekt MySQLTestData

UPDATE

Mit *UPDATE* können die Daten in den Tabellen geändert werden.

Einfache Darstellung Beim einfachen Ändern haben wir es mit wenigen Schlüsselwörter zu tun.

```
UPDATE 'Ziel' SET 'Spalte1' = 'Wert1' , 'Spalte2' = 'Wert2'
WHERE 'Where-Klausel'
```

Bei dem Ziel handelt es sich um die Tabelle in der die Daten geändert werden sollen. Nach dem Schlüsselwort *SET* erfolgt die Auflistung der Spalten mit den neuen Wert.

Wichtig ist hier die *WHERE*-Klausel! Fehlt sie so werden **alle Datensätze** der Tabelle geändert! Soll nur ein einziger Datensatz von der Änderung geändert werden, so muß die Einschränkung richtig definiert sein. Normalerweise wird hier der Primärschlüssel (auch zusammengesetzt) verwendet, denn so ist die Eindeutigkeit sichergestellt. Bezüglich der Möglichkeiten der *WHERE*-Klausel bitte beim Befehl *SELECT* nachzulesen.

Beispiele zu UPDATE

UPDATE st_person set cRName = "HoGi" WHERE STPerson = 1; Bei *UPDATE* und anderen Befehlen zur Erstellung von Daten, gibt es kein Ergebnismenge.

```
SELECT * FROM st_person;
```

```
"STPerson","cVName","cFName","cMName","cRName"
1,"Hope","Giordano","HoGi","HoGi"
2,"Harmony","Campbell","HaCa","Harmony7Campbell"
3,"Harmony","Clark","HaCl","Harmony4Clark"
4,"Holly","Collins","HoCo","Holly7Collins"
```

UPDATE st_person set cVName = "Hopper", cRName = "HoGi1" WHERE STPerson = 1; Hier werden zwei Spalten gleichzeitig geändert

```
SELECT * FROM st_person;
```

```
"STPerson","cVName","cFName","cMName","cRName"
1,"Hopper","Giordano","HoGi","HoGi1"
2,"Harmony","Campbell","HaCa","Harmony7Campbell"
3,"Harmony","Clark","HaCl","Harmony4Clark"
4,"Holly","Collins","HoCo","Holly7Collins"
```

UPDATE st_person set cVName = "HaHa" WHERE cVName like „Ha“; Hier werden mehrere Datensätze gleichzeitig geändert

```
SELECT * FROM st_person;
```

```
"STPerson","cVName","cFName","cMName","cRName"
1,"Hopper","Giordano","HoGi","HoGi1"
2,"HaHa","Campbell","HaCa","Harmony7Campbell"
3,"HaHa","Clark","HaCl","Harmony4Clark"
4,"Holly","Collins","HoCo","Holly7Collins"
```

DELETE

Mittels dem Befehl *DELETE* werden Datensätze in der Datenbank gelöscht.

Einfache Darstellung Beim einfachsten *DELETE* haben wir es mit wenigen Schlüsselwörter zu tun.

```
DELETE 'Ziel'
WHERE 'Where-Klausel'
```

Aber Vorsicht, ohne entsprechende *WHERE*-Klausel werden alle betroffenen Datensätze gelöscht. Daher gilt, fehlt die *WHERE*-Klausel, so werden **alle Datensätze** unwiderruflich gelöscht!

Beispiele zu DELETE

DELETE FROM st_person WHERE STPerson = 1; Bei *DELETE* und anderen Befehlen zur Erstellung von Daten, gibt es kein Ergebnismenge. Hier wird der Datensatz mit dem Wert 1 in der Spalte STPerson gelöscht.

```
SELECT * FROM st_person;
```

```
"STPerson","cVName","cFName","cMName","cRName"
1,"Hope","Giordano","HoGi","HoGi"
2,"Harmony","Campbell","HaCa","Harmony7Campbell"
3,"Harmony","Clark","HaCl","Harmony4Clark"
4,"Holly","Collins","HoCo","Holly7Collins"
```

```
DELETE FROM st_person WHERE STPerson = 1;
SELECT * FROM st_person;
```

```
"STPerson","cVName","cFName","cMName","cRName"
2,"Harmony","Campbell","HaCa","Harmony7Campbell"
3,"Harmony","Clark","HaCl","Harmony4Clark"
4,"Holly","Collins","HoCo","Holly7Collins"
```

DELETE st_person WHERE STPerson = 1; Bei *DELETE* und anderen Befehlen zur Erstellung von Daten, gibt es kein Ergebnismenge. Hier werden **alle** Datensätze in der Tabelle st_person gelöscht!

```
SELECT * FROM st_person;
```

```
"STPerson","cVName","cFName","cMName","cRName"
1,"Hope","Giordano","HoGi","HoGi"
2,"Harmony","Campbell","HaCa","Harmony7Campbell"
3,"Harmony","Clark","HaCl","Harmony4Clark"
4,"Holly","Collins","HoCo","Holly7Collins"
```

```
DELETE FROM st_person;
SELECT * FROM st_person;
```

```
"STPerson","cVName","cFName","cMName","cRName"
```

2.1.4 DCL Datenkontrollsprache

Die Datenkontrollsprache (Data Control Language = DCL³) umfasst die Sprachteile von Datenbanksprache, mit deren Hilfe man die Rechte vergibt, Wartungen durchführt. Es sind das Befehle wie *GRANT* und *REVOKE*.

Version: \$LastChangedRevision: 52 \$⁴

³siehe auch http://de.wikipedia.org/wiki/Data_Control_Language

⁴Autor: Andreas Frieß

3 Lazarus IDE

3.1 Entwicklungsumgebung

3.1.1 Tastenkombinationen

Die aktuelle Quelle für die Tastenkombinationen ¹ ist die Lazarus-Homepage[10].

Datei	
Tastenkürzel	Erklärung
Strg+O	Datei öffnen
Strg+S	Datei speichern
Umsch+Strg+S	alle Dateien speichern
Strg+P	Drucken

Tabelle 3.1: IDE - Tastenkombinationen Datei

Bearbeiten	
Tastenkürzel	Erklärung
Strg+Z	Rückgängig
Umsch+Strg+Z	Wiederholen
Strg+X	Auswahl ausschneiden
Strg+C	Auswahl kopieren
Strg+V	Auswahl einfügen
Strg+I	Auswahl einrücken
Strg+U	Auswahl ausrücken
Strg+A	Alles auswählen
Umsch+Strg+D	\$IFDEF einfügen
Umsch+Strg+C	Kodevervollständigung (Class Completion)

Tabelle 3.2: IDE - Tastenkombinationen Bearbeiten

Version: \$LastChangedRevision: 65 \$ ²

¹/Lazarus_IDE_Tools/de#.C3.9Cbersichtstabelle_der_IDE_Tastenkombinationen

²Autor: Andreas Frieß

Lizenz: GFDL

Suchen	
Tastenkürzel	Erklärung
F3	nächstes suchen
Umsch+F3	vorheriges suchen
Strg+R	Ersetzen
Strg+E	Inkrementielle Suche
Strg+G	Zu Zeile springen
Strg+H	Zurückspringen
Umsch+Strg+H	Vorwärtsspringen
Strg+F8	Zum nächsten Fehler springen
Umsch+Strg+F8	Zum vorherigen Fehler springen
Alt+Up	Deklaration unter Cursor suchen
Strg+Enter	Datei unter Cursor öffnen
Umsch+Strg+G	Prozedur Liste

Tabelle 3.3: IDE - Tastenkombinationen Suchen

Ansicht	
Tastenkürzel	Erklärung
F11	Objektinspektor anzeigen
Strg+F12	Unitliste anzeigen
Umsch+F12	Formularliste anzeigen
F12	Formulare/Unit Anzeige umschalten
Strg+Alt+F	Suchergebnisse anzeigen
Strg+Alt+B	Überwachte Ausdrücke
Strg+Alt+W	Haltepunkte
Strg+Alt+L	Lokale Variablen
Strg+Alt+S	Aufruf Stack

Tabelle 3.4: IDE - Tastenkombinationen Ansicht

Projekt	
Tastenkürzel	Erklärung
Strg+F11	Projekt öffnen
Umsch+Strg+F11	Projekt Einstellungen
Umsch+F11	Datei ins Projekt übernehmen

Tabelle 3.5: IDE - Tastenkombinationen Projekt

Start	
Tastenkürzel	Erklärung
Strg+F9	Erstellen
F9	Start
F7	Ein Schritt
F4	Start bis Cursor
Strg+F2	Halt
Strg+F7	Prüfen/ändern
Strg+F5	Überwachung hinzufügen

Tabelle 3.6: IDE - Tastenkombinationen Start

Divers	
Tastenkürzel	Erklärung
Strg+Click	springt zur Deklaration eines Typen oder Variablen
Strg+Shift+Upe	schaltet zwischen Definition und Rumpf um
Strg+J	Code-Schablonen
Strg+Space	Bezeichnervervollständigung
Strg+W	Word Completion
Strg+Shift+Space	Parameter Hinweise

Tabelle 3.7: IDE - Tastenkombinationen

3.1.2 Fehlersuche

Im besten Fall läuft ein Programm fehlerfrei. Da dies aber meistens bei der Entwicklung nicht so ist, gibt es viele Möglichkeiten Fehler zu suchen. Allerdings muß man sich auch über die verschiedenen Arten von Fehlern und Unzulänglichkeiten im klaren sein.

Syntax Fehler Sind Fehler in der Syntax selbst, tauchen aber oft auch als Kompilerfehler auf.

Bezeichner Namen von Objekte und Variablen mehrfach vergeben

Schlüsselwörter Schlüsselwörter als Namen von Objekten oder Variablen verwendet

Blöcke falsch Den Anfang oder das Ende von Prozedure, Funktionen oder Schleifen nicht richtig gesetzt.

Parameter falsch Typ oder Anzahl von Parametern falsch gewählt

Kompiler Fehler Sind Fehler, die beim Kompilieren auftreten, weil der Kompiler oder Präprozessor nicht in der Lage ist, den oder die Befehle zu übersetzen. oder dem Vergessen von Deklarationen.

Syntax Fehler siehe oben.

Schreibfehler Diese Fehler entstehen durch vergessene Zeileabschlüsse, fehlende Zeichen oder Buchstabendreher.

Logische Fehler Hier ist zwar die Syntax prinzipiell in Ordnung, nur in der Programmlogik hat man Fehler. Variablen vergessen zu initialisieren, Einhaltung von Abläufen nicht erzwungen, falsche Programmlogik für das Problem genommen, vergessen Objekte frei zu geben, ...

Initialisierung Fehlende oder unzureichende Initialisierung von Objekten und Variablen

Endloschleifen Schleifen werden nicht beendet

Rekursionen Ungewollte Rekursionen entstehen

Logik Ganz einfach die falsche Lösung für das richtige Problem genommen, Klammern falsch oder nicht gesetzt bei komplexen Ausdrücken

Falsche Anzahl von Durchläufen Vergessen, das 0..n nicht gleich n-Durchläufen ist.

Laufzeit Fehler Bei dieser Art, sind (Rand-) Bedingungen zur Laufzeit nicht erfüllt und der Programmierer hat keine entsprechenden Gegenmaßnahmen im Programm getroffen.

Divisionsfehler Division durch Null nicht abgefangen

Umwandlungen Inhalt von Zeichenketten enthalten keinen numerischen Wert.

Eingabe und Ausgabefehler Dateien oder Geräte die erwartet werden sind nicht oder nicht mehr verfügbar

Anhand der Liste sieht man, das es sehr viele Möglichkeiten gibt, Fehler zu machen. Ebenso gibt es viele Wege, wie man Fehler beseitigen kann. Eine der besten Möglichkeiten ist, gar keine Fehler zu machen. Klingt banal, es ist damit gemeint, das sich oft Logik und Laufzeit Fehler durch Planung stark vermindern lassen. Dazu gehört auch, das man Teilproblemstellungen im Vorfeld testet, ob der Lösungsansatz für das Problem wirklich der Richtige ist.

DebugLn

Ausgabe von Informationen an die Debugkonsole.

HeapTrace

Ist ein Werkzeug bei der Entdeckung von Speicherlöcher und Leichen. Zu finden ist das im Menü unter Projekt -> Compilereinstellungen -> Linker Punkt 'heaptrc unit verwenden'.

Version: \$ \$ ³

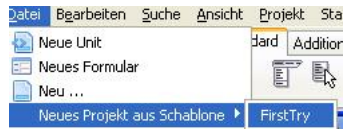
³ Autor: Andreas Frieß

Lizenz: GFDL

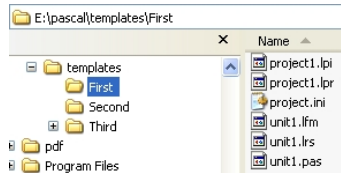
3.1.3 Projektschablonen

Beschreibung

Projektschablonen sind Vorlagen für öfters verwendete Projekte. Das Komponente „projekttemplates“ hilft bei der Verwendung von Schablonen⁴. Als Vorbereitung muß man die Komponente installieren.



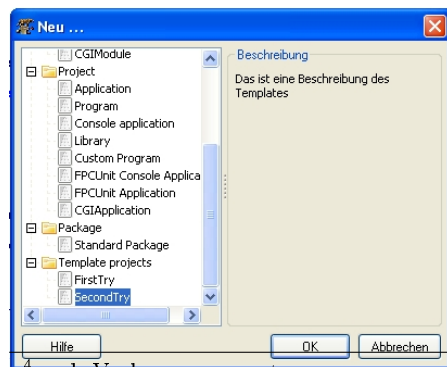
Nach der Erstellung ist das Menü „Werkzeuge“ um den Eintrag „Optionen für Projektschablonen“ erweitert. Mit Hilfe dieses Menüpunktes kann man das Basis Verzeichnis für die Schablonen einstellen. Die Schablonen selbst befinden sich dann in den Unterverzeichnissen. Weiters wird auch das Menü „Datei“ um den Eintrag „Neues Projekt aus Schablone“ erweitert. Zusätzlich ist auch der „Datei Neu ..“ Dialog entsprechend erweitert. Jede Schablone liegt in einem eigenen Unterverzeichnis und kann selbst weitere Unterverzeichnisse enthalten. Diese Struktur wird beim Erstellen eines neuen Projektes mittels Schablonen übertragen⁵.



Die Steuerung der Vorlagen erfolgt durch die Datei „project.ini“. Diese Datei ist im „ini“ Format aufgebaut und es befinden sich derzeit zwei Sektionen darinnen. In der ersten Sektion mit dem Namen „[Variables]“ befinden sich die Variablen⁶. Als Voreinstellung selbst gibt es zwei vordefinierte Variablen die die Ersetzungsroutine selbst kennt und die nicht definiert werden müssen.

ProjDir Das Verzeichnis, in welches das neue Projekt erzeugt wird

ProjName Der Name des neuen Projektes



⁴ auch Vorlagen genannt

⁵ Es mir in der derzeitigen Version Lazarus 0.9.25 svn 13811 nicht geglückt das die Unterverzeichnisse kopiert werden

⁶ Platzhalter, veränderbare Teile

⁷ Wurde gegenüber der Beschreibung im README.txt offenbar geändert - war \$(VARNAME)

den Namen der Variable zu ersetzen. Die Ersetzung wird beim Kopieren in das neue Projekt sowohl in den Dateinamen gemacht, als auch innerhalb der Dateien.

Die zweite Sektion beschäftigt sich mit dem beschreiben des Projektes selbst. Die Sektion „[Project]“ beinhaltet folgende Informationen zum Projekt, wobei der Namen, Author und die Beschreibung in Datei-Neu Dialog von Lazarus angezeigt wird.

Name Namen der Vorlage

Author Author der Vorlage

Description Kurze Beschreibung der Vorlage, maximal einzeilig

Recurse Ob in Unterverzeichnis hineinrekursiert⁸ werden soll (0=Nein, 1=Ja)

Exclude Durch Komma getrennte Liste, welche Dateien nicht für die Ersetzung verwendet werden sollen

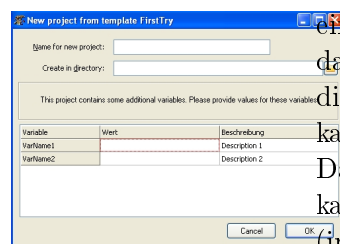
Ist in dem Verzeichnis eine Datei mit dem Namen „description.txt“, so wird der Inhalt der Datei als Beschreibung verwendet.

Beispiel

Als Beispiel habe ich ein vorhandenes einfaches Projekt im Vorlagen Verzeichnis abgespeichert. Weiter kommt in diese Verzeichnis eine „project.ini“ Datei mit folgen Inhalt hinein

```
[Variables]
VarName1= Dateiname fuer das Hauptformular
[Project]
Name=SecondTry
Author= Nobody
Description= Test template 1
Recurse= 0
Exclude=
```

In der ersten Eingabezeile wird der Dateiname des Projektes eingegeben, er ist in der nicht sichtbaren Variablen „ProjName“ dann vorhanden. In der zweiten Zeile gibt man den Pfad, wohin die Dateien aus den Vorlagen kopiert werden sollen an. Das kann entweder durch Direkteingabe erfolgen oder über den Dateidialog, den man rechts neben der Eingabezeile aufrufen kann. Darunter befindet sich dann die Eingabemaske für die frei (in der „project.ini“) definierbaren Variablen. Der ausgefüllte Inhalt der Variablen ersetzt dann beim kopieren die Platzhalter.



⁸funktioniert ?!

3 Lazarus IDE

Heisst also eine Datei im Templateverzeichnis zum Beispiel `__VarName1__.pas` und bei der Variablen wird „myform“ festgelegt, so wird die Datei als *myform.pas* in das Zielverzeichnis kopiert. Ebenso verhält es sich mit Variablen (nicht verwechseln mit Variablen innerhalb von Lazarus), die beim kopieren ebenso durch die Werte ersetzt werden.

Wird der Dialog jetzt bestätigt, so wird die Kopieraktion durchgeführt und Lazarus öffnet das neue Projekt.

Version: \$ \$ ⁹

⁹ Autor: Andreas Frieß
Lizenz: GFDL

Literaturverzeichnis

- [LazDe] Deutsches Lazarusforum <http://www.lazarusforum.de> Forum
- [LazEn] Englischs Lazarusforum <http://www.lazarus.freepascal.org>
- [FPCLangRef] Reference guide for Free Pascal, version 2.0.4, Document version 2.0
August 2006 <ftp://ftp.freepascal.org/pub/fpc/docs-pdf/ref.pdf>
- [10] Lazarus Wiki <http://www.lazarus.freepascal.org> Übersichts-
tabelle_der_IDE_Tastenkombinationen
- [11] Homepage <http://www.ullihome.de/>
- [12] Homepage <http://homepages.codegear.com/jedi/jcl/>
- [14] Homepage <http://www.?????/?/?/?/?/>

3.2 GNU Free Documentation License

Version 1.2, November 2002

Copyright © 2000,2001,2002 Free Software Foundation, Inc.

51 Franklin St, Fifth Floor, Boston, MA 02110-1301 USA

Everyone is permitted to copy and distribute verbatim copies of this license document, but changing it is not allowed.

Preamble

The purpose of this License is to make a manual, textbook, or other functional and useful document “free” in the sense of freedom: to assure everyone the effective freedom to copy and redistribute it, with or without modifying it, either commercially or noncommercially. Secondly, this License preserves for the author and publisher a way to get credit for their work, while not being considered responsible for modifications made by others.

This License is a kind of “copyleft”, which means that derivative works of the document must themselves be free in the same sense. It complements the GNU General Public License, which is a copyleft license designed for free software.

We have designed this License in order to use it for manuals for free software, because free software needs free documentation: a free program should come with manuals providing the same freedoms that the software does. But this License is not limited to software manuals; it can be used for any textual work, regardless of subject matter or whether it is published as a printed book. We recommend this License principally for works whose purpose is instruction or reference.

1. APPLICABILITY AND DEFINITIONS

This License applies to any manual or other work, in any medium, that contains a notice placed by the copyright holder saying it can be distributed under the terms of this License. Such a notice grants a world-wide, royalty-free license, unlimited in duration, to use that work under the conditions stated herein. The “**Document**”, below, refers to any such manual or work. Any member of the public is a licensee, and is addressed as “**you**”. You accept the license if you copy, modify or distribute the work in a way requiring permission under copyright law.

A “**Modified Version**” of the Document means any work containing the Document or a portion of it, either copied verbatim, or with modifications and/or translated into another language.

A “**Secondary Section**” is a named appendix or a front-matter section of the Document that deals exclusively with the relationship of the publishers or authors of the Document to the Document’s overall subject (or to related matters) and contains nothing that could fall directly within that overall subject. (Thus, if the Document is in part a textbook of mathematics, a Secondary Section may not explain any mathematics.) The relationship

could be a matter of historical connection with the subject or with related matters, or of legal, commercial, philosophical, ethical or political position regarding them.

The “**Invariant Sections**” are certain Secondary Sections whose titles are designated, as being those of Invariant Sections, in the notice that says that the Document is released under this License. If a section does not fit the above definition of Secondary then it is not allowed to be designated as Invariant. The Document may contain zero Invariant Sections. If the Document does not identify any Invariant Sections then there are none.

The “**Cover Texts**” are certain short passages of text that are listed, as Front-Cover Texts or Back-Cover Texts, in the notice that says that the Document is released under this License. A Front-Cover Text may be at most 5 words, and a Back-Cover Text may be at most 25 words.

A “**Transparent**” copy of the Document means a machine-readable copy, represented in a format whose specification is available to the general public, that is suitable for revising the document straightforwardly with generic text editors or (for images composed of pixels) generic paint programs or (for drawings) some widely available drawing editor, and that is suitable for input to text formatters or for automatic translation to a variety of formats suitable for input to text formatters. A copy made in an otherwise Transparent file format whose markup, or absence of markup, has been arranged to thwart or discourage subsequent modification by readers is not Transparent. An image format is not Transparent if used for any substantial amount of text. A copy that is not “Transparent” is called “**Opaque**”.

Examples of suitable formats for Transparent copies include plain ASCII without markup, Texinfo input format, LaTeX input format, SGML or XML using a publicly available DTD, and standard-conforming simple HTML, PostScript or PDF designed for human modification. Examples of transparent image formats include PNG, XCF and JPG. Opaque formats include proprietary formats that can be read and edited only by proprietary word processors, SGML or XML for which the DTD and/or processing tools are not generally available, and the machine-generated HTML, PostScript or PDF produced by some word processors for output purposes only.

The “**Title Page**” means, for a printed book, the title page itself, plus such following pages as are needed to hold, legibly, the material this License requires to appear in the title page. For works in formats which do not have any title page as such, “Title Page” means the text near the most prominent appearance of the work’s title, preceding the beginning of the body of the text.

A section “**Entitled XYZ**” means a named subunit of the Document whose title either is precisely XYZ or contains XYZ in parentheses following text that translates XYZ in another language. (Here XYZ stands for a specific section name mentioned below, such as “**Acknowledgements**”, “**Dedications**”, “**Endorsements**”, or “**History**”.) To “**Preserve the Title**” of such a section when you modify the Document means that it remains a section “Entitled XYZ” according to this definition.

The Document may include Warranty Disclaimers next to the notice which states that this License applies to the Document. These Warranty Disclaimers are considered to be included by reference in this License, but only as regards disclaiming warranties: any other implication that these Warranty Disclaimers may have is void and has no effect on

the meaning of this License.

2. VERBATIM COPYING

You may copy and distribute the Document in any medium, either commercially or noncommercially, provided that this License, the copyright notices, and the license notice saying this License applies to the Document are reproduced in all copies, and that you add no other conditions whatsoever to those of this License. You may not use technical measures to obstruct or control the reading or further copying of the copies you make or distribute. However, you may accept compensation in exchange for copies. If you distribute a large enough number of copies you must also follow the conditions in section 3.

You may also lend copies, under the same conditions stated above, and you may publicly display copies.

3. COPYING IN QUANTITY

If you publish printed copies (or copies in media that commonly have printed covers) of the Document, numbering more than 100, and the Document's license notice requires Cover Texts, you must enclose the copies in covers that carry, clearly and legibly, all these Cover Texts: Front-Cover Texts on the front cover, and Back-Cover Texts on the back cover. Both covers must also clearly and legibly identify you as the publisher of these copies. The front cover must present the full title with all words of the title equally prominent and visible. You may add other material on the covers in addition. Copying with changes limited to the covers, as long as they preserve the title of the Document and satisfy these conditions, can be treated as verbatim copying in other respects.

If the required texts for either cover are too voluminous to fit legibly, you should put the first ones listed (as many as fit reasonably) on the actual cover, and continue the rest onto adjacent pages.

If you publish or distribute Opaque copies of the Document numbering more than 100, you must either include a machine-readable Transparent copy along with each Opaque copy, or state in or with each Opaque copy a computer-network location from which the general network-using public has access to download using public-standard network protocols a complete Transparent copy of the Document, free of added material. If you use the latter option, you must take reasonably prudent steps, when you begin distribution of Opaque copies in quantity, to ensure that this Transparent copy will remain thus accessible at the stated location until at least one year after the last time you distribute an Opaque copy (directly or through your agents or retailers) of that edition to the public.

It is requested, but not required, that you contact the authors of the Document well before redistributing any large number of copies, to give them a chance to provide you with an updated version of the Document.

4. MODIFICATIONS

You may copy and distribute a Modified Version of the Document under the conditions of sections 2 and 3 above, provided that you release the Modified Version under precisely this License, with the Modified Version filling the role of the Document, thus licensing distribution and modification of the Modified Version to whoever possesses a copy of it. In addition, you must do these things in the Modified Version:

- A. Use in the Title Page (and on the covers, if any) a title distinct from that of the Document, and from those of previous versions (which should, if there were any, be listed in the History section of the Document). You may use the same title as a previous version if the original publisher of that version gives permission.
- B. List on the Title Page, as authors, one or more persons or entities responsible for authorship of the modifications in the Modified Version, together with at least five of the principal authors of the Document (all of its principal authors, if it has fewer than five), unless they release you from this requirement.
- C. State on the Title page the name of the publisher of the Modified Version, as the publisher.
- D. Preserve all the copyright notices of the Document.
- E. Add an appropriate copyright notice for your modifications adjacent to the other copyright notices.
- F. Include, immediately after the copyright notices, a license notice giving the public permission to use the Modified Version under the terms of this License, in the form shown in the Addendum below.
- G. Preserve in that license notice the full lists of Invariant Sections and required Cover Texts given in the Document's license notice.
- H. Include an unaltered copy of this License.
- I. Preserve the section Entitled "History", Preserve its Title, and add to it an item stating at least the title, year, new authors, and publisher of the Modified Version as given on the Title Page. If there is no section Entitled "History" in the Document, create one stating the title, year, authors, and publisher of the Document as given on its Title Page, then add an item describing the Modified Version as stated in the previous sentence.
- J. Preserve the network location, if any, given in the Document for public access to a Transparent copy of the Document, and likewise the network locations given in the Document for previous versions it was based on. These may be placed in the "History" section. You may omit a network location for a work that was published at least four years before the Document itself, or if the original publisher of the version it refers to gives permission.

- K. For any section Entitled “Acknowledgements” or “Dedications”, Preserve the Title of the section, and preserve in the section all the substance and tone of each of the contributor acknowledgements and/or dedications given therein.
- L. Preserve all the Invariant Sections of the Document, unaltered in their text and in their titles. Section numbers or the equivalent are not considered part of the section titles.
- M. Delete any section Entitled “Endorsements”. Such a section may not be included in the Modified Version.
- N. Do not retitle any existing section to be Entitled “Endorsements” or to conflict in title with any Invariant Section.
- O. Preserve any Warranty Disclaimers.

If the Modified Version includes new front-matter sections or appendices that qualify as Secondary Sections and contain no material copied from the Document, you may at your option designate some or all of these sections as invariant. To do this, add their titles to the list of Invariant Sections in the Modified Version’s license notice. These titles must be distinct from any other section titles.

You may add a section Entitled “Endorsements”, provided it contains nothing but endorsements of your Modified Version by various parties—for example, statements of peer review or that the text has been approved by an organization as the authoritative definition of a standard.

You may add a passage of up to five words as a Front-Cover Text, and a passage of up to 25 words as a Back-Cover Text, to the end of the list of Cover Texts in the Modified Version. Only one passage of Front-Cover Text and one of Back-Cover Text may be added by (or through arrangements made by) any one entity. If the Document already includes a cover text for the same cover, previously added by you or by arrangement made by the same entity you are acting on behalf of, you may not add another; but you may replace the old one, on explicit permission from the previous publisher that added the old one.

The author(s) and publisher(s) of the Document do not by this License give permission to use their names for publicity for or to assert or imply endorsement of any Modified Version.

5. COMBINING DOCUMENTS

You may combine the Document with other documents released under this License, under the terms defined in section 4 above for modified versions, provided that you include in the combination all of the Invariant Sections of all of the original documents, unmodified, and list them all as Invariant Sections of your combined work in its license notice, and that you preserve all their Warranty Disclaimers.

The combined work need only contain one copy of this License, and multiple identical Invariant Sections may be replaced with a single copy. If there are multiple Invariant

Sections with the same name but different contents, make the title of each such section unique by adding at the end of it, in parentheses, the name of the original author or publisher of that section if known, or else a unique number. Make the same adjustment to the section titles in the list of Invariant Sections in the license notice of the combined work.

In the combination, you must combine any sections Entitled “History” in the various original documents, forming one section Entitled “History”; likewise combine any sections Entitled “Acknowledgements”, and any sections Entitled “Dedications”. You must delete all sections Entitled “Endorsements”.

6. COLLECTIONS OF DOCUMENTS

You may make a collection consisting of the Document and other documents released under this License, and replace the individual copies of this License in the various documents with a single copy that is included in the collection, provided that you follow the rules of this License for verbatim copying of each of the documents in all other respects.

You may extract a single document from such a collection, and distribute it individually under this License, provided you insert a copy of this License into the extracted document, and follow this License in all other respects regarding verbatim copying of that document.

7. AGGREGATION WITH INDEPENDENT WORKS

A compilation of the Document or its derivatives with other separate and independent documents or works, in or on a volume of a storage or distribution medium, is called an “aggregate” if the copyright resulting from the compilation is not used to limit the legal rights of the compilation’s users beyond what the individual works permit. When the Document is included in an aggregate, this License does not apply to the other works in the aggregate which are not themselves derivative works of the Document.

If the Cover Text requirement of section 3 is applicable to these copies of the Document, then if the Document is less than one half of the entire aggregate, the Document’s Cover Texts may be placed on covers that bracket the Document within the aggregate, or the electronic equivalent of covers if the Document is in electronic form. Otherwise they must appear on printed covers that bracket the whole aggregate.

8. TRANSLATION

Translation is considered a kind of modification, so you may distribute translations of the Document under the terms of section 4. Replacing Invariant Sections with translations requires special permission from their copyright holders, but you may include translations of some or all Invariant Sections in addition to the original versions of these Invariant Sections. You may include a translation of this License, and all the license notices in the Document, and any Warranty Disclaimers, provided that you also include the original English version of this License and the original versions of those notices and disclaimers.

In case of a disagreement between the translation and the original version of this License or a notice or disclaimer, the original version will prevail.

If a section in the Document is Entitled “Acknowledgements”, “Dedications”, or “History”, the requirement (section 4) to Preserve its Title (section 1) will typically require changing the actual title.

9. TERMINATION

You may not copy, modify, sublicense, or distribute the Document except as expressly provided for under this License. Any other attempt to copy, modify, sublicense or distribute the Document is void, and will automatically terminate your rights under this License. However, parties who have received copies, or rights, from you under this License will not have their licenses terminated so long as such parties remain in full compliance.

10. FUTURE REVISIONS OF THIS LICENSE

The Free Software Foundation may publish new, revised versions of the GNU Free Documentation License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns. See <http://www.gnu.org/copyleft/>.

Each version of the License is given a distinguishing version number. If the Document specifies that a particular numbered version of this License “or any later version” applies to it, you have the option of following the terms and conditions either of that specified version or of any later version that has been published (not as a draft) by the Free Software Foundation. If the Document does not specify a version number of this License, you may choose any version ever published (not as a draft) by the Free Software Foundation.

ADDENDUM: How to use this License for your documents

To use this License in a document you have written, include a copy of the License in the document and put the following copyright and license notices just after the title page:

Copyright © YEAR YOUR NAME. Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 1.2 or any later version published by the Free Software Foundation; with no Invariant Sections, no Front-Cover Texts, and no Back-Cover Texts. A copy of the license is included in the section entitled “GNU Free Documentation License”.

If you have Invariant Sections, Front-Cover Texts and Back-Cover Texts, replace the “with ... Texts.” line with this:

Literaturverzeichnis

with the Invariant Sections being LIST THEIR TITLES, with the Front-Cover Texts being LIST, and with the Back-Cover Texts being LIST.

If you have Invariant Sections without Cover Texts, or some other combination of the three, merge those two alternatives to suit the situation.

If your document contains nontrivial examples of program code, we recommend releasing these examples in parallel under your choice of free software license, such as the GNU General Public License, to permit their use in free software.